CONSUMER INSIGHT PORTAL

How the Forsta ETL works

From a survey in Forsta to verified, report-ready numbers in our database. Each stage is explained step by step, with every technical word defined where it first appears.

① Source: Forsta ② Schedule ③ API ④ Data received ⑤ Extract
⑥ Transform ⑦ Load ⑧ Verify ⑨ Derive ⑩ Failures

Version 1.0 · 13 October 2026 · Facts taken from the code (`etl/`, `scripts/`) and the live database

How to read this document

Each **keyword** ⓘ *a technical term you may not know* is **bold with a yellow underline**, and its meaning follows straight after it in a yellow note, like this one. You never need to turn to a glossary.

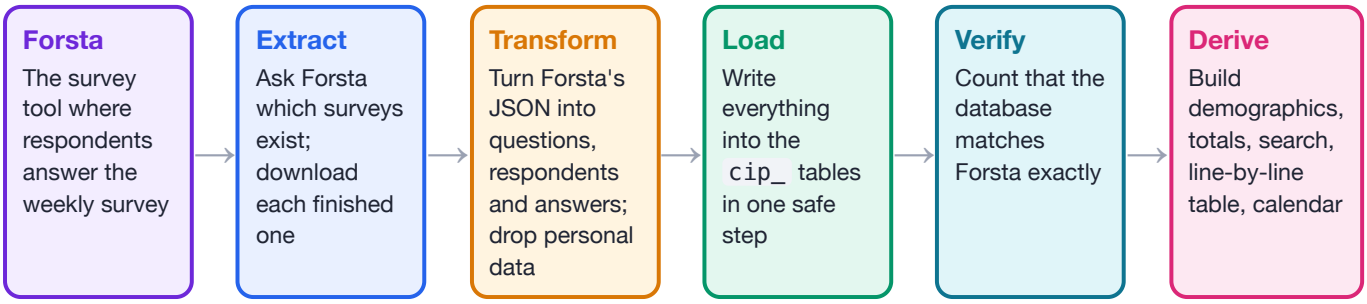
Source Extract Transform Load Verify Derive Failure

Each stage keeps the same colour in every heading, diagram and table.

1. The whole pipeline on one page
2. Where the data comes from: Forsta
3. When it runs: frequency and timing
4. The Forsta API: what we call
5. What data we receive
6. Extract: finding and fetching surveys
7. Transform: turning JSON into rows
8. Load: writing to the database
9. Verify: proving nothing was lost
10. Derive: making it report-ready
11. What we ingest and what we never store
12. When something goes wrong
13. Worked example: one wave, start to finish

1. The whole pipeline on one page

ETL ⓘ *Extract, Transform, Load: take data out of one system, reshape it, and put it into another* is the program that copies our consumer survey answers from Forsta into our database. It runs on its own, several times a day.



Stage	What actually happens	Code
Source	Respondents from a panel answer a survey in Forsta. When the target of about 400 qualified people is reached, the survey is **closed**.	—
Extract	Gets the list of every survey. Each survey is marked *waiting*, *due*, *loaded* or *hibernated*. For each *due* survey it downloads the questionnaire and every response.	`etl/forsta_etl.py` `etl/forsta_client.py`
Transform	Works out each question's type (single choice, tick-all, grid, number, text). Converts every answer from text to the right type. Removes or scrambles personal fields.	`etl/forsta_api.py` `etl/records.py`
Load	Writes the survey, its questions, its respondents and their answers. Respondents and answers are written together, so they either all succeed or all fail.	`etl/loaders.py`
Verify	Checks that the numbers of respondents, questions and answers match what Forsta sent. Only then does the wave appear in reports.	`verify()` in `etl/forsta_etl.py`
Derive	Builds age and income bands, regions, cross-wave question links, pre-counted totals, search, the data team's line-by-line table and the survey calendar.	`etl/run_pipeline.py` `app/data/`

In one sentence

Several times a day the job asks Forsta "what's new and finished?". It downloads each new survey in full, reshapes it and stores it. It then counts that nothing was lost before anyone can see it.

2. Where the data comes from: Forsta

Forsta ⓘ *the survey platform, formerly called Decipher, where Coresight's surveys are written, run and stored; our account is on the server se1.decipherinc.com* holds every survey Coresight runs. Respondents never touch our systems; they answer in Forsta.

2.1 The things Forsta holds

Thing	What it is	Example
Survey	One questionnaire run once. For the weekly tracker, **one survey = one wave**. **Wave** ⓘ *one week's run of the same survey; waves are compared week to week*	"Shopping and Spending – inc Home Furnishings + Holiday Tracker + GenAI"
Path	**Path** ⓘ *Forsta's unique address for a survey, in the form selfserve/58f/YYMMNN, where 58f is the Coresight folder* . It is the key we use to track each survey.	`selfserve/58f/260908`
Respondent / record	One person's attempt at the survey. **Record** ⓘ *Forsta's word for one respondent's row of answers*	record 412, uuid `a1b2…`
Status	How far the person got: **1** terminated (screened out), **2** over quota (the group was already full), **3** qualified (finished and counted), **4** partial (left before the end).	Weekly wave: ≈400 qualified out of 403–887 started

Thing	What it is	Example
Datamap	**Datamap** ⓘ *Forsta's machine-readable description of the questionnaire: every question code, its wording, its type and every answer option with its number*	`D1` "What is your gender?" 1=Male, 2=Female…
State	The survey's lifecycle: *testing* → *live* (in field) → *closed*.	`"state": "closed"`
Hibernated	**Hibernated** ⓘ *Forsta moves old closed surveys (about 3–4 months after close) to cold storage; their data cannot be read until a person reactivates them in Forsta*. The API then answers **428**.	The 2025 holiday waves

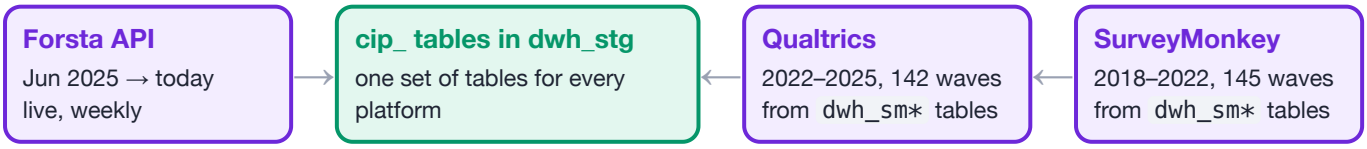
2.2 The surveys

Kind	Pattern
Weekly tracker	"Shopping and Spending – inc …". Launched every **Monday**. It has 15 core questions (q1–q5 activities and purchases, CS1–CS2 outlook, D1–D8 demographics) plus that week's modules.
Module	**Module** ⓘ *an extra block of questions added to a wave, such as a retail sector or a topic like tariffs or GLP-1*. The lead sector rotates every 12 weeks: Drugstore → eCommerce → Luxury → Dept stores → Beauty → Home furnishings → Home improvement → Social commerce → Off-price → Apparel → Mass/warehouse → Footwear.
Holiday tracker	Questions coded `XM`, added to every weekly wave from late September to December. Waves are numbered within each season.
Annual / ad hoc	Annual studies (holiday shopping, back to school, online grocery…) and one-off client studies. Same pipeline, same tables.

2.3 Our access

We read Forsta with an **API key** ⓘ *a 64-character secret password for programs, stored in the git-ignored .env file and never printed or logged*. It belongs to user 879 in the "Coresight Research" folder (`58f`). The key can **only read**: our code has no way to edit, reactivate or delete anything in Forsta.

2.4 Other sources (history only)



dwh_stg ⓘ *our Azure MySQL staging database; all pipeline tables in it start with cip_*. History was loaded once. Only Forsta is fetched on a schedule.

3. When it runs: frequency and timing

The job is started by **launchd** ⓘ *the macOS built-in scheduler, which runs a program at set times like cron on a server; if the Mac was asleep, it runs the missed slot when it wakes* through the agent `com.coresight.cip-forsta-etl`, which runs `scripts/weekly_forsta.py`.

3.1 The schedule (India time, IST)

Day	09:00	14:00	23:30	Why
Monday	✅			Daily safety run
Tuesday	✅	✅	✅	Most waves close Monday night to Tuesday
Wednesday	✅	✅	✅	Late waves close by Wednesday 23:00
Thu – Sun	✅			Picks up ad hoc studies and reopened surveys

That is **13 runs a week**. Most find nothing to do and finish in seconds. On a server the same schedule is cron `0 9 * * *`, `0 14 * * 2,3`, `30 23 * * 2,3`.

3.2 A typical week, measured over 66 weekly waves

Mon 17:30–18:30 IST
Survey launches (12–13 UTC) in 56 of 66 waves

→

In field
Median 13 hours to reach ≈400 qualified; 90% within 49 h

→

Closed
Between Mon 20:00 and Wed 23:00 IST

→

Next run
Tue/Wed 09:00, 14:00 or 23:30 finds it *due*

→

In reports
1.5–3 minutes later, verified

So how fresh is the data?

A wave is usually in the database within **a few hours of closing**, and at most about 10–15 hours later (the gap between two runs). A wave is **never** loaded while it is still *live*, because it would be incomplete.

3.3 What one run does, in order

1. **Database health check.** `SELECT VERSION()`. If the database cannot be reached, it tries 4 times, 5 minutes apart, then stops with **exit code** ⓘ *a number a program returns when it ends: 0 means fine and anything else names the problem (see section 12)* 4.
2. **Key check.** `GET rh/users/self`. If Forsta rejects the key, it stops with exit 2.
3. **Schema check.** The newest tables (`cip_forsta_survey`, `cip_search`, `cip_tracker_line`) must exist. If not, exit 2. **Schema** ⓘ *the definition of the database tables and columns*
4. **Discover** every survey and mark what is due (section 6).
5. **Extract → Transform → Load → Verify → Derive** each due wave, oldest first (sections 6–10).
6. **Drift check.** Recompute every frozen published number. If one moved, exit 3. **Drift** ⓘ *a number already published to clients changing because the underlying data changed*

Everything is appended to `logs/forsta-etl.log`.

4. The Forsta API: what we call

An **API** ⓘ *Application Programming Interface: web addresses that return data to programs instead of web pages to people* is how we read Forsta. Ours is Forsta's **REST** ⓘ *a style of API where each address is a "resource" and you use standard web verbs such as GET to read it* API, version 1.

4.1 Connection settings

Setting	Value and meaning
Base URL	`https://se1.decipherinc.com/api/v1/`. Every call starts with this. **HTTPS** means the traffic is encrypted.
Method	**GET only.** **GET** ⓘ *the web verb that only reads; POST, PUT and DELETE would create, change or delete, and our client does not contain them*
Authentication	Header `x-apikey: <key>`. **Header** ⓘ *extra information sent with each request, alongside the address*. Also sent: `Accept: application/json` and `User-Agent: coresight-cip/1.0`.
Format	**JSON** ⓘ *JavaScript Object Notation, a text format of names and values in {braces} and [lists], readable by people and programs*
Timeout	120 seconds per request (`FORSTA_TIMEOUT_SECONDS`). If Forsta does not answer in that time, the request is retried.
Retries	Up to **4 attempts**, waiting longer each time, from 2 s up to 60 s (**exponential back-off** ⓘ *wait, then wait about twice as long before each new try, so a busy server gets breathing room*). Retried on connection errors, timeouts, 429 and 5xx.
Usage	Forsta's `x-usage-today` header (calls used today) is written to the debug log.
Client code	`etl/forsta_client.py`, class `ForstaClient`

4.2 The four endpoints, the only ones we use

An **endpoint** ⓘ *one specific API address that returns one kind of data*:

Endpoint (all GET)	Returns	How often
`rh/users/self`	Who the key belongs to. Used only to prove the key works.	1 per run
`rh/companies/all/surveys`	Every survey the key can see, with path, title, state, hibernated, tags, dates and counts (about 90 surveys).	1 per run
`surveys/{path}/datamap?format=json`	The questionnaire definition (≈0.5 MB).	1 per wave loaded
`surveys/{path}/data?format=json`	Every respondent's answers (≈2 MB per 500 respondents). We send **no** `cond` filter, so all statuses come back: terminated, over quota, qualified and partial.	1 per wave loaded; 1 per wave in a reconciliation

Calls per run

A run with nothing due makes **2 calls**. A run that loads one weekly wave makes **4 calls**: key, list, datamap and data. The survey list is fetched once and reused for every wave in the run.

4.3 What Forsta's answers mean

Every reply carries an **HTTP status code** ⓘ *a three-digit number with every web reply: 2xx success, 4xx a problem with our request, 5xx a problem on the server* :

Code	Meaning	What the pipeline does
200	OK	Carries on.
401	Key missing, wrong, revoked, or the user is disabled	Stops the run, exit 2. A person must renew the key.
403	Key valid but no rights on this survey	This wave fails and is recorded; the others continue.
428	Survey hibernated	Never loaded. Left alone; reconciliation skips it with a warning.
429	Too many requests (**rate limit** ⓘ *the server's cap on calls per period*)	Waits and retries, up to 4 attempts.
5xx	Forsta server error	Retries up to 4 times. If it still fails, the wave is marked *failed* and retried at the next run.

5. What data we receive

Forsta sends three kinds of JSON. Below is one real example of each; comments in yellow are ours.

5.1 The survey list: one item per survey

```
{ "path": "selfserve/58f/260908",        // unique survey address
  "title": "Shopping and Spending – inc Home Furnishings + Holiday Tracker + GenAI",
  "state": "closed",                       // testing | live | closed
  "hibernated": false,                     // true = data not readable
  "tags": ["Coresight Research", "Weekly consumer tracker"],
  "createdOn":    "2026-09-22T15:57:00Z",    // Z = UTC time
  "dateLaunched": "2026-10-05T12:22:25Z",
  "closedDate":   "2026-10-06T14:19:55Z",
  "qualified": 401,  "total": 823,         // finished & counted / everyone who started
  "sampleSources": ["114"] }               // the respondent panel used
```

We use this to decide **what to download**. It is stored in the register table `cip_forsta_survey`. A **register** ⓘ *one row per Forsta survey recording what Forsta says about it and what the pipeline has done with it*

5.2 The datamap: the questionnaire

```
{ "questions": [
  { "qlabel": "q1",                          // question code
    "qtitle": "Which, if any, of these have you done in the past two weeks?",
    "type": "multiple",                      // single | multiple | number | float | text
    "grouping": "rows",
    "variables": [                          // one per answer column in the data
      { "label": "q1r1", "rowTitle": "Met up with friends", "type": "multiple" },
      { "label": "q1r2", "rowTitle": "Eaten at a restaurant", ... } ] },
  { "qlabel": "D1", "qtitle": "What is your gender?", "type": "single",
    "values": [ {"value": 1, "title": "Male"}, {"value": 2, "title": "Female"}, ... ] },
  { "qlabel": "qtime", ..., "flags": ["t"] } ] }   // t = technical, v = virtual
```

A **variable** ⓘ *one column in Forsta's data; a single-choice question has one, a tick-all question has one per option (q1r1, q1r2…)* is the unit we store answers against. **Flags** ⓘ *markers on a question: "t" technical (timings, status, conditions) and "v" virtual (values Forsta computes, e.g. device or browser)* tell us which questions are not real survey questions.

5.3 The data: one object per respondent

```
[ { "record": "412", "uuid": "8k2f…",            // Forsta's ids for this attempt
    "status": "3",                              // 3 = qualified
    "date": "10/05/2026 14:07",                 // completion, MM/DD/YYYY HH:MM
    "qtime": "611",                             // seconds taken
    "q1r1": "1", "q1r2": "0",                 // ticked / shown but not ticked
    "D1": "2", "D2": "34", "D8": "TX",        // codes, numbers, text: ALL strings
    "q9oe": "Bought a new sofa",                // open-ended text
    "RID": "…", "userAgent": "…", "url": "…", "session": "…", "dcua": "…"  // personal: dropped or hashed
  }, ... ]
```

Important detail

Every value arrives as text, even numbers and codes (`"3"`, `"34"`). Part of the transform is converting each value to its proper type. An empty value means the question was not asked or not answered.

5.4 Size of a typical weekly wave

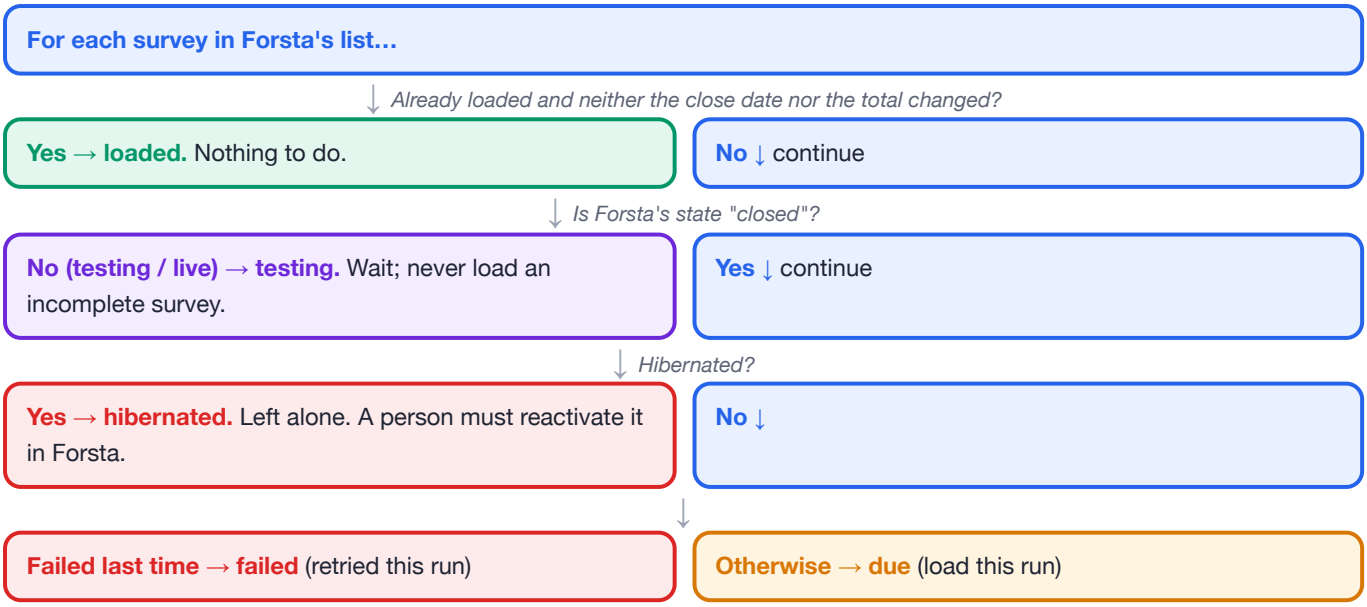
Measure	Typical value
Respondents in the payload (all statuses)	403 – 887 (≈400 of them qualified)
Questions per wave	≈60 (1,416 measured across the loaded waves)
Answer rows stored per wave	≈145,000 on average (4,476,153 across 31 Forsta waves)
Download size	datamap ≈0.5 MB + data ≈2 MB
Load time	1.5 – 3 minutes, mostly ≈250 ms database round trips from India to Azure

6. Extract: finding and fetching surveys

What happens: the pipeline first lists everything, decides which surveys are ready, then downloads each ready one in full.

6.1 Step 1: discovery, on every run

Discovery ⓘ *the step that asks Forsta for the full survey list and updates the register; it downloads no answers* calls `rh/companies/all/surveys` once and **upserts** ⓘ *"update or insert": change the row if it exists, add it if not, so running twice never creates duplicates* one row per survey into `cip_forsta_survey`, keyed by the path. It then gives each survey a **load state** ⓘ *the pipeline's own label for a survey: testing, due, loaded, failed or hibernated*:



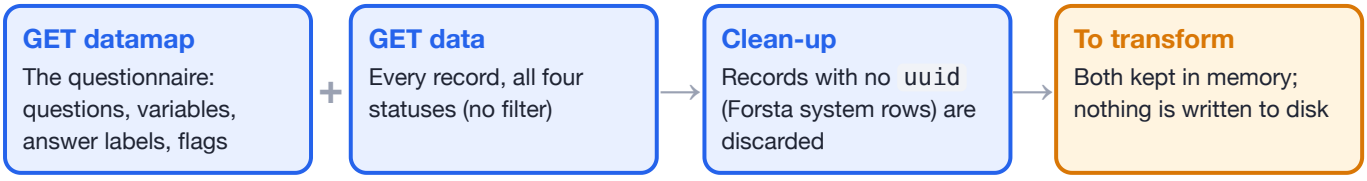
Reopened surveys: if a loaded survey's close date or total changes (it was reopened and more people answered), it becomes *due* again and is reloaded in place.

Discovery also records `study_type`: **tracker** if the survey is tagged "Weekly consumer tracker" or titled "Shopping and spending…", **annual** if tagged "annual tracker", otherwise **adhoc**. In the same step it rebuilds the survey calendar (section 10).

6.2 Step 2: choose what to load

All *due* and *failed* surveys are taken **oldest close date first** and loaded **one at a time**. If one fails, its error is saved in `last_error` and the run moves on to the next. One bad survey never blocks the others.

6.3 Step 3: download the wave



Why fetch every status, not only qualified? Partial and terminated records show where people dropped out and let us check our counts against Forsta's `total`. Reports count **qualified respondents only**.

6.4 Step 4: work out which week it is

Item	Rule
Wave date / label	The **Monday** of the earliest completion date among **qualified** records (all records if none qualified). The wave launched Mon 5 Oct 2026 becomes `2026-10-05`.
Reload	A survey this pipeline has loaded before (load log `forsta:{path}`) keeps its wave label and internal `survey_id`, even if its first date moves.
Family	`CSI-US` for the weekly tracker. For other studies, the title without the year in capitals (`HOLIDAY_SHOPPING`). **Family** ⓘ *the group of waves that are compared with each other over time*

7. Transform: turning JSON into rows

What happens: the datamap becomes **question definitions** and each record becomes **one respondent row plus one answer row per answered variable**. Personal data is removed at this point, before anything reaches the database.



7.1 Recognising the question type

Forsta's `type` and `grouping` decide how a question is stored. Each question becomes our `qtype`:

Forsta shape	Seen	Stored as	Meaning and example
`single`, 1 variable	693	**single**	**Single choice** ⓘ *pick exactly one answer*. D1 gender: one field `D1`, value 1–4.
`multiple`	491	**multi**	**Tick-all / checkbox** ⓘ *pick any number of items; each item is its own yes/no column*. q1 activities: fields `q1r1`, `q1r2` … each 0 or 1.
`single`, rows, many variables	73	**grid_single**	**Grid** ⓘ *several statements (rows), each answered on the same scale (columns)*. One field per row; row label "row — column".
`single` + a text variable	53	**single** + **text**	A choice question with an "Other (please specify)" box. The box becomes its own text question (see 7.2).
`single`, rows, 1 variable	43	single or grid	Grid if the variable's name differs from the question code.
`number` / `float`	32	**numeric**	A typed number, such as an amount spent.
`text` / `textarea`	23	**text**	**Verbatim** ⓘ *the respondent's own words, stored exactly as typed*.

7.2 The rules applied to every question

#	Rule	What it does
1	Clean wording	HTML codes such as `&` become normal characters and extra spaces are removed. A short label of 80 characters or fewer is made for charts.
2	Answer labels	Taken from the question's `values`. In about a third of datamaps these are empty, so they are taken from the first variable that has them. Only whole-number codes are kept.
3	Grid detection	Spots a single-choice question that has several rows.
4	Bipolar grid	**Bipolar grid** ⓘ *each row is a pair of opposite statements and the respondent picks one side, e.g. "I'll shop early" vs "I'll shop late"*. Forsta leaves the answer titles blank, so we store 1 = left statement and 2 = right statement, and save both statements per row. The old Excel route lost about 3,300 of these answers.
5	Other (specify)	The text box inside a choice question becomes its own text question, e.g. `q1r2oe`, worded "<question> — <row> (please specify)".

#	Rule	What it does
6	Technical / virtual	Flag `t` (timings, status, conditions) or `v` (device, browser, panel list), or a title ending "-NORMALIZED". These are **stored but never reported**.
7	Personal fields	`RID`, `userAgent`, `url`, `session`, `dcua` and `ipAddress` never become questions, even if the datamap lists them.
8	Topic	Each question is placed in an editorial topic (demographics, holiday, tariffs…) from its code and wording, using `config/survey_map.yml`.
9	Answer flags	"Don't know / Not sure / Prefer not to say" are marked **non-response**. "None of these" is marked **exclusive**. Item codes ending `oe` are marked **other**.

7.3 Each record → one respondent row

We store	From Forsta	Rule
`record_no`, `forsta_uuid`	`record`, `uuid`	Forsta's own ids, used as the key so reloads update rather than duplicate.
`status_code`, `status_label`, `is_qualified`	`status`	1 Terminated · 2 Overquota · 3 Qualified · 4 Partial. Qualified = status 3.
`completed_at`	`date`	`MM/DD/YYYY HH:MM` text → a real date-time.
`interview_secs`	`qtime`	Whole seconds.
`device`, `os`, `browser`, `dropout_qcode`	`vmobiledevice`, `vos`, `vbrowser`, `vdropout`	Short category labels, at most 50 characters (not the raw browser string).
`panel_source`	`vlist` / `list`	Which panel supplied the person.
`respondent_key`	`RID`	**SHA-256 hash** ⓘ *a one-way scramble: the same input always gives the same 64-character code, but the code cannot be turned back into the panel id*. The real panel id is never stored.

7.4 Each answered variable → one answer row

Answers are stored **long, not wide** ⓘ *one row per person per variable, instead of one column per question; the table never needs new columns when weekly modules change* in `cip_answer`:

Variable kind	Stored as
Code (single, grid row, checkbox item)	`value_code` as a whole number (`"3"` and `"3.0"` both become 3), plus `value_label`, e.g. "Female".
Code that is not a whole number	The raw text goes in `value_label`; no code is invented.
Number	`value_number`, only if it is a real finite number below 10^{14} (the column holds 4 decimals).
Text	`value_text`, exactly as typed.
Empty	**No row.** "Not answered" means no row exists.
Checkbox "0"	**Stored.** It means "shown but not ticked", and it gives every tick-all question its base of people who answered.

Example: one record becomes rows

Forsta value	field	value_code	value_number	value_text / label
`"D1": "2"`	D1	2		Female
`"q1r1": "1"`	q1r1	1		Met up with friends
`"q1r2": "0"`	q1r2	0		Eaten at a restaurant (not ticked)
`"D2": "34"`	D2	34		34
`"q9oe": "Bought a new sofa"`	q9oe			Bought a new sofa
`"q7": ""`	*no row: not asked or not answered*			
`"userAgent": "Mozilla…"`	*dropped: personal*			

8. Load: writing to the database

What happens: the transformed wave is written into `dwh_stg` in a fixed order. The step that replaces respondents and answers is a single **transaction** ⓘ *a group of database writes that either all succeed together or are all undone together, so nobody ever sees half a wave* .

1 **Survey row** in `cip_survey` , set to `load_status = loading` (not visible in reports).

↓

2 **Run log** row in `cip_load_log` , with `source_ref = forsta:{path}` . This is the audit trail of every run. **Load log** ⓘ *a diary of every load: what, when, rows read and rows written, success or error*

↓

3 **Definitions**: topics, questions, options, items and fields (`cip_topic` , `cip_question` , `cip_option` , `cip_item` , `cip_field`). One batched statement per table.

↓

4 **Transaction A, all or nothing:**
a) upsert every respondent into `cip_respondent` , keyed by survey + record;
b) delete respondents Forsta no longer sends, with their answers, profile and cohort rows;
c) delete the wave's old answers, then insert the new ones in batches of 5,000 rows.

↓

5 **Verify** (section 9). If anything is off, the wave is marked *failed*.

↓

6–11 **Derive**: profiles, question bases, concepts, cohorts and cube, search, line-by-line (section 10).

↓

12–14 **Finish**: log row closed with row counts. `cip_survey.load_status = verified` , register set to *loaded*, calendar rebuilt.

8.1 Why it is safe to run again and again

Idempotent

Idempotent ⓘ *running the same load twice gives exactly the same result as running it once*. Every write is an upsert or a replace keyed on Forsta's own ids (path, uuid, record, variable label), so nothing is ever duplicated.

No half-loaded waves

If transaction A fails, the previous answers are untouched. A wave that was already verified stays visible with its old, correct data.

Why batches?

Each database call from India to Azure takes about 250 ms. Writing 145,000 answers one by one would take 10 hours. Batches of 5,000 keep a wave to a fixed, small number of calls, which is why a wave loads in 1.5–3 minutes.

8.2 Where it lands: the main tables

Table	One row per	Per weekly wave
`cip_forsta_survey`	Forsta survey (the register)	1
`cip_survey`	Wave	1
`cip_question` / `cip_option` / `cip_item` / `cip_field`	Question / answer option / grid row or checkbox item / Forsta variable	≈60 questions, hundreds of fields
`cip_respondent`	Person who started	403 – 887
`cip_answer`	Person × variable answered	≈145,000
`cip_load_log`	Load run	1 per load

9. Verify: proving nothing was lost

What happens: before a wave can appear anywhere, the pipeline re-reads what it stored and compares it with the payload it downloaded. **Payload** ⓘ *the exact data Forsta sent in this run*

① Respondents
Stored respondents for the wave = records sent

+

② Fields
Every datamap variable (except personal ones) has a field

+

③ Answers
For every field: stored answers = records with a non-empty value

→

All equal
Status `verified`; visible

Any difference

The error "`{path} does not match its payload`" is saved in `cip_load_log.error_text` and `cip_forsta_survey.last_error`. The wave is marked **failed**, stays hidden and is retried at the next run.

9.1 Visibility

load_status	Meaning	In reports?
loading	Being written	No
verified	Proven equal to Forsta, with everything derived	**Yes**
failed	Did not match, or a later step failed	No

9.2 Reconciliation: an independent second check

Reconciliation ⓘ *re-downloading every loaded wave from Forsta and comparing every single stored answer with it* is run with `python scripts/reconcile.py`. It compares codes exactly, numbers to 4 decimals and texts exactly. Hibernated surveys are skipped with a warning.

Last result (8 Oct 2026)

31 Forsta waves · 4,476,153 answers · 0 differences. The 142 Qualtrics and 145 SurveyMonkey waves are checked against their `dwh_sm*` source the same way.

10. Derive: making it report-ready

What happens: after verification, everything a report needs is calculated **once, at load time**, so the portal and exports only read finished numbers.

Output (table)	What it is and how it is built
Profile `cip_profile`	One row per respondent with their demographics, read from D1 gender, D2 age, D3 relationship, D4 ethnicity, D5 income, D6 urban/suburban/rural, D7 politics, D8 state, CS1/CS2 outlook. Adds: • **Age band**: 18-29 · 30-44 · 45-60 · Over 60 ("Under 18" is not banded) • **Generation**: Gen Z 18-28 · Millennial 29-44 · Gen X 45-60 · Boomer 61+ • **Income midpoint** ($000): <$25k = 13 · $25–49.9k = 37.5 · $50–99.9k = 75 · $100–149.9k = 125 · $150–199.9k = 175 · $200k+ = 225 • **Census region**: state → Northeast / Midwest / South / West
Question base `cip_question.base_n`	**Base** ⓘ *the number of people a percentage is calculated over: qualified respondents who were shown the question*
Concepts `cip_concept*`	**Concept** ⓘ *"the same question" across waves and platforms, matched by wording because Forsta codes such as D2 or BT14 change meaning between waves*. Same wording and options → confirmed automatically. Changed options, or wording at least 85% similar → proposed for a person to confirm. Otherwise → a new concept.
Cube `cip_agg_cell`	**Cube** ⓘ *a table of pre-counted totals: for every answer, how many people chose it, broken down by each standard cut*. Cuts: total, gender, age band, generation, income band, region, urbanicity, each for everyone and for each **cohort** ⓘ *a defined group of respondents, e.g. "Beauty shoppers"*. Qualified respondents only.
Search `cip_search`	A full-text index of wave titles, question wording and concepts, so "GLP-1" finds every wave that asked about it.
Line-by-line `cip_tracker_line`	The data team's "Weekly Line-By-Line Survey Data" sheet, rebuilt automatically: one row per qualified respondent in the team's column order. Checked against the team's file across 16 waves: identical apart from 3 errors in the team's file.

Output (table)	What it is and how it is built
Survey calendar `cip_wave_module`	Which sector, topics and seasonal trackers each wave carried, plus the holiday-season wave number. This enables "wave N vs last year's wave N" and predicts the next 12 lead sectors.

11. What we ingest and what we never store

Ingested

Data	From
Every survey's metadata: title, state, tags, dates, counts	survey list
Every question, answer option, grid row and variable	datamap
Every respondent of **every status** (1–4)	data
Every non-empty answer: codes, numbers, open text	data
Completion time, interview length, device / OS / browser category, dropout question, panel source	data
Technical and virtual questions (stored, hidden)	datamap + data

Never stored

Data	Why
`RID` (panel id)	Personal; stored only as a SHA-256 hash
`userAgent`, `dcua`	Browser fingerprint
`url`, `session`	Survey link and panel session
`ipAddress`	Personal
Raw JSON files	Kept in memory only; nothing written to disk
Surveys still *testing* / *live*	Incomplete; loaded once closed
Hibernated surveys	Unreadable (428) until a person reactivates them

11.1 Where things stand (13 Oct 2026)

Source	Waves verified	Notes
Forsta (live)	31	4,476,153 answers, 0 differences. Wave `261000` (12 Oct, Home improvement + Holiday Tracker) is still live in Forsta and will load automatically once it closes.
Qualtrics (history)	142	2022–2025
SurveyMonkey (history)	145	2018–2022

12. When something goes wrong

Every run ends with an exit code, written to the log, that says how it went:

Code	Meaning	What to do
0	Everything due loaded and verified, or nothing was due	Nothing
1	A wave failed to load or verify	Read `cip_forsta_survey.last_error`. The next run retries it automatically.

Code	Meaning	What to do
2	Forsta rejected the key, the key is missing, or the database schema is older than the code	Renew the key in `.env`, or run `python scripts/init_db.py`
3	Loaded fine, but an already-published number changed	Review on the Publications page
4	Database unreachable after 4 tries over 15 minutes	Check the VPN or the Azure firewall (the office IP changed on 13 Oct). The next slot retries.

12.1 Common situations

You see	Cause	Fix
"Loaded 0" for days	No survey has closed in Forsta	Expected
"Can't connect to MySQL … timed out"	VPN down, or the office IP is not allowed in Azure	Reconnect; the next slot retries
A wave "does not match its payload"	Forsta changed data mid-load, or a step broke	`python -m etl.forsta_etl --survey <path>`
Survey *hibernated* and its history is needed	Forsta hibernated it before it was loaded	A Forsta owner reactivates it; the next run loads it
Survey reopened and closed again	New respondents	Automatic: becomes due and is reloaded in place

12.2 Commands to run by hand

```
python scripts/weekly_forsta.py                          # the scheduled run, now
python -m etl.forsta_etl --discover                      # refresh the register only (no answers
downloaded)
python -m etl.forsta_etl --due                           # discover + load everything due
python -m etl.forsta_etl --survey selfserve/58f/260908   # load or reload one wave
python -m etl.forsta_etl --reconcile                     # compare every loaded wave with Forsta
python scripts/test_connection.py                        # database + Forsta check (never prints the
key)
```

13. Worked example: one wave, start to finish

The wave `selfserve/58f/260908`, "Shopping and Spending – inc Home Furnishings + Holiday Tracker + GenAI":

Tue 22 Sep 2026. Created in Forsta. Discovery sees it as *testing*, so it waits.

↓

Mon 5 Oct, 12:22 UTC (17:52 IST). Launched and *live*. Runs still mark it *testing*; nothing is downloaded.

↓

Tue 6 Oct, 14:19 UTC (19:49 IST). Closed with 401 qualified out of 823 started.

↓

Next run after close (in normal operation, Tue 6 Oct 23:30 IST; this wave was actually loaded on 8 Oct while the pipeline was being built). Health ✓, key ✓, schema ✓. Discovery: closed, not hibernated, never loaded → **due**.

↓

Extract. GET datamap (≈0.5 MB) + GET data (823 records, all statuses). Rows with no uuid are discarded.

↓

Transform. 54 reportable questions typed: core q1–q6, CS1–CS3, D1–D8 plus extra D codes, HF1–HF7 home furnishings, XM1–XM3 holiday tracker, G1–G8 GenAI. Six "Other (specify)" boxes (e.g. `HF2r28oe`) become their own text questions. Personal fields are dropped and RID is hashed. Every value is converted from text to its type.

↓

Load. Survey row (*loading*) → log row → definitions → transaction A (823 respondents, 187,468 answers in 5,000-row batches). Took 2 min 20 s. Wave label = Monday **2026-10-05**.

↓

Verify. 823 = 823 respondents; every variable has a field; every field's answer count matches → ✓.

↓

Derive. Profiles and bands → bases → concepts → cube → search → 401 line-by-line rows → calendar: lead sector Home furnishings (6th in the rotation), GenAI topic, Holiday tracker **2026 wave 2**.

↓

Done. `verified`, register *loaded*, exit 0. The Holiday Databank numbers (53.9 / 37.2 / 8.2 …) now come straight from this wave.